Build and Use a Planning Canvas

Session 23 | Product and Delivery Teams | Intermediate

Use visible state when chat stops being enough

A canvas helps when work has several records, repeated updates, named actions, and a review view that must survive the conversation.

The source is the live Session 22 parent issue and its linked child issues, not the previous chat.

The canvas is a planning projection

GitHub owns issue state. The canvas groups that state with planning information that helps people decide what to do next.

GitHub fact	Canvas planning view
Open issue	Ready for discovery
Assigned issue	Ownership confirmed
Blocked issue	Risk, dependency, and next decision
Closed issue	Planning result recorded

Label the two kinds of state differently. Otherwise the team cannot tell whether they are seeing a decision or a source-system fact.

Keep fields that change a decision

Field	Keep it when	Remove it when
Risk	It changes sequencing or approval	It only repeats the issue body
Owner	The responsible person is unclear	GitHub already makes it clear
Next action	A team needs a follow-up	It is only a status synonym

The canvas should expose important work without becoming a second backlog.

Steering needs a review loop

```
Observe source state
      ↓
Propose canvas update
      ↓
Review the affected record
      ↓
Accept, revise, or reject
```

This loop protects against stale data and confident but unsupported updates.

Start from live GitHub issues

The canvas should show:

- parent objective;
- issue and planning status;
- owner or ownership gap;
- dependency and risk;
- decision and next action.

Create it in plain language

```
/create-canvas Create a delivery-planning canvas for the approved GitHub issues.
People can update planning status and record decisions. Copilot can summarize
and propose updates. Do not close issues, change assignees, or publish content.
```

This is a no-code review. Inspect generated files and capabilities, then ask Copilot for bounded changes.

Review before use

Ask:

1. Does each field support a real decision?
2. Can every action be explained?
3. Are external writes excluded?
4. Can the owner recover?
5. When should the canvas be retired?

Simpler is better

Remove generated fields and actions that do not help the team decide or act.

A crowded canvas is not a mature canvas.

Load live GitHub issues

Use GitHub MCP to retrieve the Session 22 issues.

Copilot can move the approved fields into the canvas. GitHub remains the source for issue state.

Keep planning state explicit

The canvas may show `ready for review` while the GitHub issue remains open.

That is acceptable when the canvas labels the planning state and source-system state separately.

Verify three views

Compare:

- the visible canvas;
- Copilot's report;
- a fresh GitHub MCP read.

Resolve or explain every mismatch.

Create visible drift

1. Preview and approve a GitHub issue comment through MCP.
2. Retrieve the changed issue in a fresh request.
3. Do not refresh the canvas.
4. Compare the new GitHub state with the visible canvas.

The canvas should show that its last refresh is older than the source change.

Reconcile the projection

Use the approved refresh capability.

Confirm:

- the blocker appears in planning state;
- the next action names the owner decision;
- GitHub state and planning state stay distinct;
- the last-refresh time changes.

Critique the live state

Run `/context`, then use a fresh session when needed.

Ask `/rubber-duck` to find stale claims, mixed state labels, unsafe actions, and missing owners.

The live canvas and linked issues are authoritative for Session 24.

Lab result

- Learner-created canvas
- Reviewed state and actions
- Live GitHub issue data
- A detected and reconciled state mismatch
- Owner, refresh, recovery, and retirement rules
- Four linked status-visibility workstream candidates

Stop if live canvas creation is unavailable.